UNIVERSITY OF ASIA PACIFIC

Department of Computer Science and Engineering

Course Title: Database Systems Lab

Course Code: CSE - 212

Project Name: Online Shopping Management System

Submitted By-

Name : Md. Abdullah,
Shovon Debnath,
Sabbir Ahmed Shisir

Reg. No. : 23201140,
23201142,
23201163

Semester : 2nd Year 2nd semester

Section : C-2

Submitted To-

Alif Ruslan

Lecturer,

Department of Computer Science and Engineering,

University of Asia Pacific

Table of Contents

Project Description: Online Shopping Management System 3
Entities 4
Tables Used in the Project 4
1. Customers 4
2. Category 4
3. Sellers 5
4. Products 5
5. Orders 5
6. Cointains 6
7. Inventory 6
8. Shipping 6
9. Payments 7
Relationship Between Entities 7
1) Customers ↔ Orders 7
2) Category ↔ Products 7
3) Sellers ↔ Products 7
4) Orders ↔ Products (via Contains) 8
5) Products ↔ Inventory 8
6) Orders ↔ Shipping 8
7) Orders ↔ Payments 8
8) Sellers ↔ Orders 9
ER Diagram 9
Schema Diagram 10
Queries 11
Easy Queries 11
Basic Filtering & Conditions 11
Basic Updates 12
Basic Sorting 13
Basic Aggregate Functions 13
Moderate Queries 14
Multi-table Joins 14
Aggregation with GROUP BY 14
Hard Queries 17
Correlated Subqueries 17
Subqueries inside WHERE 17
Comparing Aggregates with Global Aggregates 18
Distinct Count Subqueries 18
Group-Based Filtering Using Subqueries 19
Conclusion 19
CEP Mapping 21

Project Description: Online Shopping Management System

The Online Shopping Management System is a relational database project designed to manage the essential backend operations of an e-commerce platform. Online shopping involves handling large volumes of data about customers, products, sellers, orders, payments, and deliveries. Without a proper structure, this data can easily become inconsistent, duplicated, or difficult to manage.

To solve this, the project uses a well-designed MySQL database (implemented in OneCompiler) with tables such as Customers, Category, Products, Sellers, Orders, Inventory, Shipping, Payments, and a junction table Contains for handling multi-product orders. Each table includes appropriate primary and foreign keys to maintain integrity and accurate relationships.

The system supports:

1. Managing customer information, including addresses and phone numbers
2. Organizing products under categories and linking them with sellers
3. Recording and tracking customer orders with support for multiple products
4. Monitoring stock updates through the Inventory table
5. Processing online payments and storing payment status
6. Tracking shipping progress and managing delivery/return details

The database follows normalization principles to reduce redundancy and improve efficiency. Its modular design allows easy expansion with features like reviews, discounts, or seller analytics. Built and tested on OneCompiler, the project also showcases how cloud-based tools simplify development and rapid prototyping.

Entities

Customers: (customer_id, name, district, address, phone)
Orders: (order_id, order_date, *customer_id*, order_status, *product_id*, *seller_id*)
Payments: (payment_id, payment_date, payment_mode, payment_status, *order_id*)
Inventory: (inventory_id, *product_id*, stock_remaining, warehouse_id, restock_date)
Category: (category_id, category_name)
Products: (product_id, product_name, price, *category_id*, *seller_id*)
Shipping: (shipping_id, *order_id*, delivery_status, shipping_date, return_date)
Sellers: (seller_id, seller, name)

Tables Used in the Project

1. Customers

customer_id (Primary Key)
name (Not Null)
district
address
phone

This table stores detailed information about customers who place orders on the online shopping platform. It helps identify users, their location for delivery, and their contact number for communication. Every order placed in the system is linked to a record in this table, making it a core component of customer-related operations.

2. Category

category_id (Primary Key)
category_name

This table maintains the list of product categories available in the system such as electronics, clothing, groceries, etc. Categorizing products ensures easier browsing, filtering, and management of inventory. Each product in the system

must be assigned to exactly one category, establishing a structured product classification.

3. Sellers

seller_id (Primary Key)
seller_name

This table contains information about all sellers registered on the platform. Sellers supply the products and are responsible for fulfilling orders. The table helps identify the source of each product and supports seller-based analytics and filtering.

4. Products

product_id (Primary Key)
product_name
price
category_id (Foreign Key)
seller_id (Foreign Key)

The Products table stores all item details available for purchase. Each product includes fields such as name, price, category, and seller information.
The **category_id** ensures that every product belongs to a valid category, and the **seller_id** links each product to the seller offering it. This table is central to search, ordering, and inventory operations.

5. Orders

order_id (Primary Key)
order_date
customer_id (Foreign Key)
order_status
product_id (Foreign Key)
seller_id (Foreign Key)

This table records all purchase orders made by customers. It includes the date of purchase, the customer who placed the order, and the status (pending, delivered, cancelled).
Each entry also specifies the product purchased and the seller fulfilling the

order. By referencing Customers, Products, and Sellers, this table enables tracking of order histories and fulfillment progress.

6. Cointains

order_id (Primary Key, Foreign Key)
product_id (Primary Key, Foreign Key)

This table manages the many-to-many relationship between orders and products. Since an order can include multiple products and a product can appear in multiple orders, this intermediate table ensures proper linkage. It also allows future expansion for quantities or per-item discounts.

7. Inventory

inventory_id (Primary Key)
product_id (Foreign Key)
stock_remaining
warehouse_id
restock_date

This table keeps track of stock availability for each product. It helps maintain warehouse records, including how many units are left and when the item was last restocked. By referencing the Products table, the system ensures accurate synchronization between product listings and real-time stock levels.

8. Shipping

shipping_id (Primary Key)
order_id (Foreign Key)
delivery_status
shipping_date
return_date

This table Tracks shipping information of orders, including shipping date and return status. Each order may have one associated shipping record.

9. Payments

payment_id (Primary Key)
payment_date
payment_mode
payment_status
order_id (Foreign Key)

This table manages payment records for the orders placed. It includes the mode of payment (cash on delivery, card, digital wallet), the date of payment, and the payment confirmation status. The order_id foreign key ensures that each payment corresponds to a valid order.

Relationship Between Entities

1) Customers ↔ Orders

Relationship: One-to-Many
Explanation:
A customer can place multiple orders over time, but each order is linked to only one customer. This relationship helps track complete purchase histories for each customer.

2) Category ↔ Products

Relationship: One-to-Many
Explanation:
A category may contain several products, while each product belongs to only one category. This supports organized product classification throughout the system.

3) Sellers ↔ Products

Relationship: One-to-Many
Explanation:

A seller can list multiple products, but every product is associated with only one seller. This enables the system to track product ownership and seller inventory.

4) Orders ↔ Products (via Contains)

Relationship: Many-to-Many
Explanation:
Orders can include multiple products, and a product can be part of many orders. The Contains table serves as the junction entity, linking each product with the orders in which it appears. This design allows multi-item orders and enables detailed tracking of product sales across many customers.

5) Products ↔ Inventory

Relationship: One-to-Many
Explanation:
A single product may have multiple inventory entries, especially if stocked in different warehouses or updated over time. The Inventory table records stock levels, warehouse IDs, and restock dates, enabling effective inventory monitoring for each product.

6) Orders ↔ Shipping

Relationship: One-to-Many
Explanation:
An order may generate multiple shipping records—for example, separate shipments, returns, or delivery updates. Each shipping entry refers back to one order through the order_id. This structure gives flexibility for tracking complex delivery workflows.

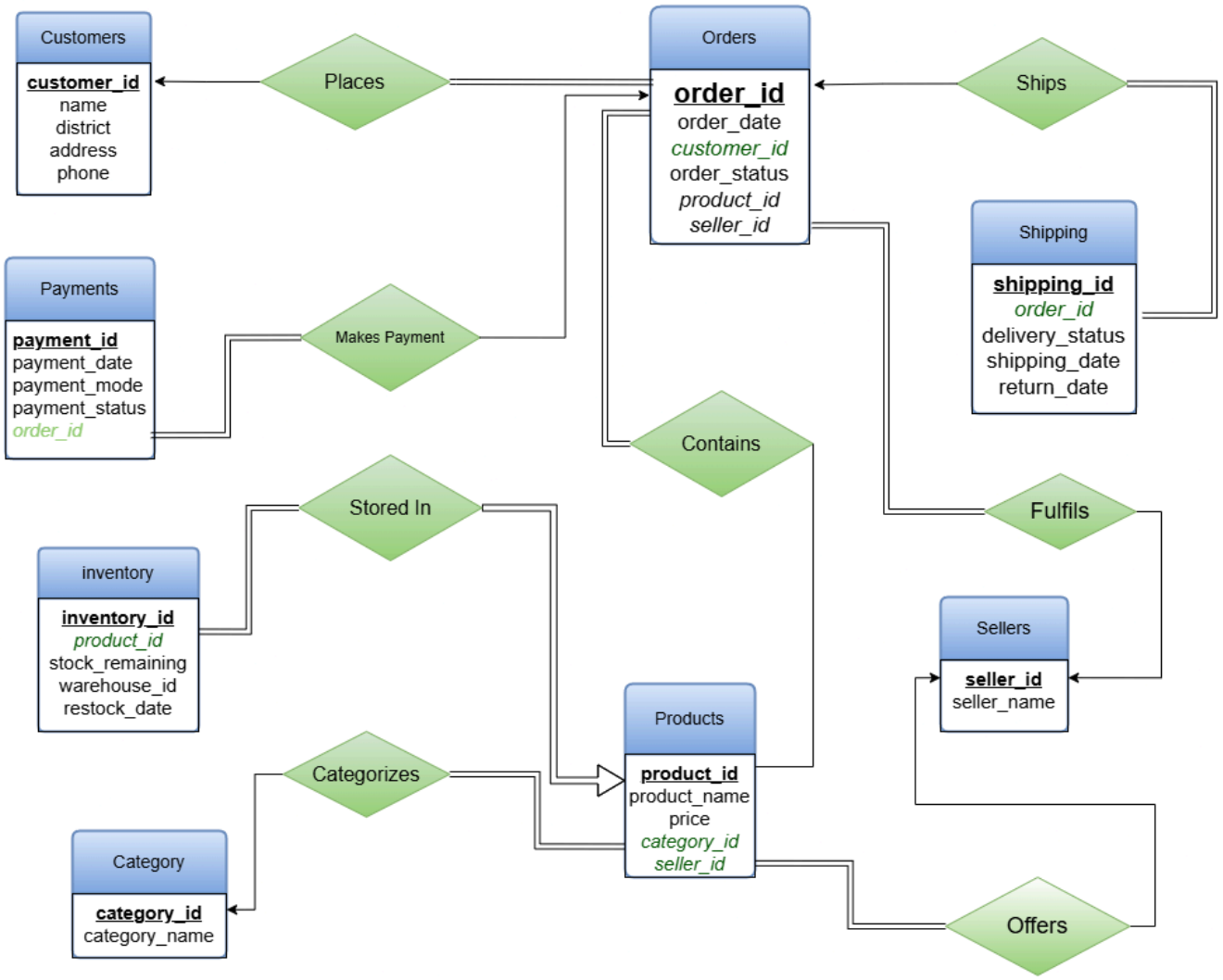
7) Orders ↔ Payments

Relationship: One-to-Many
Explanation:
A single order may have multiple payment entries—for instance, partial payments, failed attempts, or refunds. Each payment record is linked to an order through order_id, enabling detailed tracking of different payment events linked to the same order.

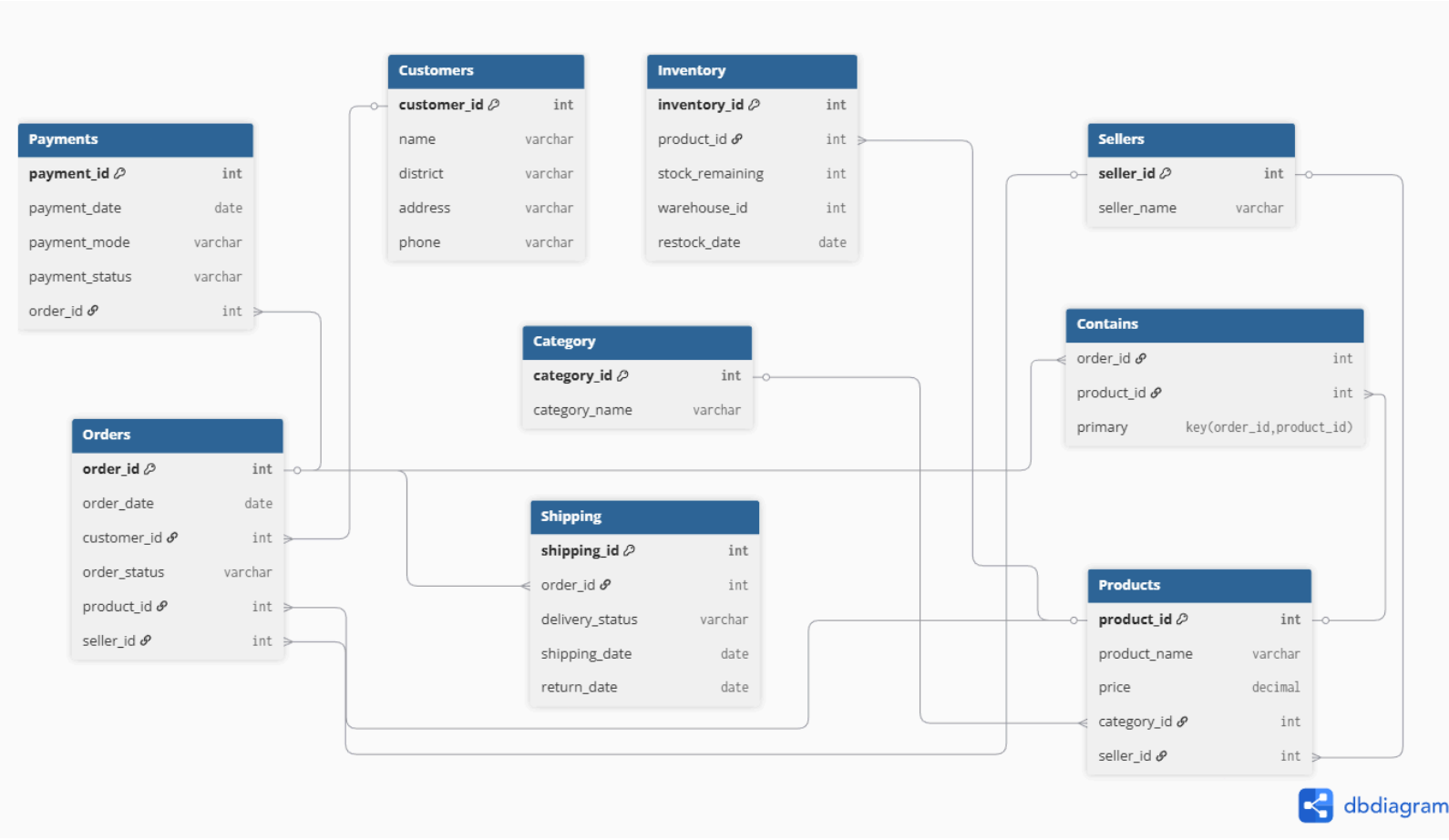
8) Sellers ↔ Orders

Relationship: One-to-Many
Explanation:
A seller may handle multiple orders depending on the products they supply. The Orders table stores seller_id, identifying which seller fulfilled that particular order. This relationship supports seller performance analysis and order distribution reporting.

ER Diagram



Schema Diagram



Queries

Easy Queries

Basic Filtering & Conditions

1. **Products priced between 300 and 800**

```
SELECT product_name, price
FROM Products
WHERE price BETWEEN 300 AND 800
LIMIT 5;
```

product_name	price
Watch	510.09
Backpack	728.78
Car Tire	535.56
Flour	361.44
Headphones	614.37

2. **Customers from Dhaka or Sylhet**

```
SELECT name, district
FROM Customers
WHERE district = 'Dhaka' OR district = 'Sylhet'
LIMIT 5;
```

name	district
Sabbir Ahmed	Dhaka
Md Abdullah	Dhaka
Shovon Debnath	Dhaka
Rahim Uddin	Dhaka
Nusrat Jahan	Sylhet

3. **Orders that are not delivered**

```
SELECT order_id, order_status
FROM Orders
WHERE NOT order_status = 'Delivered'
LIMIT 5;
```

order_id	order_status
100	Pending
101	Processing
102	Shipped
105	Processing
106	Pending

Basic Updates

4. **Updating inventory stock**

```
UPDATE Inventory
SET stock_remaining = stock_remaining + 20
WHERE product_id = 105;
Select * FROM Inventory
LIMIT 5;
```

inventory_id	product_id	stock_remaining	warehouse_id	restock_date
100	111	40	5	2022-02-13
101	111	108	5	2022-02-28
102	104	68	5	2020-01-03
103	108	26	9	2021-09-22
104	110	136	5	2025-08-04

Basic Sorting

5. **Products ordered by price (descending)**

```
SELECT product_name, price
FROM Products
ORDER BY price DESC
LIMIT 5;
```

product_name	price
Doll	961.70
Camera	887.56
Laptop	841.81
Toy Car	838.05
Pants	781.39

Basic Aggregate Functions

6. **SUM, AVG, COUNT on Products**

```
SELECT
     SUM(price) AS Total_Product_Price,
     AVG(price) AS Average_Product_Price,
     COUNT(product_id) AS Total_Products
FROM Products;
```

Total_Product_Price	Average_Product_Price	Total_Products
10104.70	505.235000	20

Moderate Queries

Multi-table Joins

7. **Top 3 customers by total order value**

```
SELECT
    c.customer_id,
    c.name AS customer_name,
    SUM(p.price) AS total_order_value
FROM Orders o
JOIN Contains co ON o.order_id = co.order_id
JOIN Products p ON co.product_id = p.product_id
JOIN Customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_order_value DESC
LIMIT 3;
```

customer_id	customer_name	total_order_value
119	Samiul Hossain	2774.76
106	Nusrat Jahan	1946.56
114	Hasibul Hasan	1336.55

Aggregation with GROUP BY

8. **Min and Max product price per category**

```
SELECT cat.category_name, MIN(p.price) AS min_price,
MAX(p.price) AS max_price
FROM Products p
JOIN Category cat ON p.category_id = cat.category_id
GROUP BY cat.category_name
LIMIT 5;
```

category_name	min_price	max_price
Electronics	887.56	887.56
Books	321.85	781.39
Home & Kitchen	247.71	535.56
Beauty & Personal Care	378.27	378.27
Grocery	838.05	838.05

9. **Count how many orders each customer placed**

```
SELECT c.name AS customer_name, COUNT(o.order_id) AS
total_orders
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.name
ORDER BY total_orders DESC
LIMIT 5;
```

customer_name	total_orders
Md Abdullah	4
Samiul Hossain	2
Farzana Karim	2
Rafiul Islam	2
Nusrat Jahan	2

10. **Total products sold by each seller**

```
SELECT
    s.seller_id,
    s.seller_name,
    COUNT(co.product_id) AS total_products_sold
FROM Sellers s
JOIN Products p ON s.seller_id = p.seller_id
JOIN Contains co ON p.product_id = co.product_id
JOIN Orders o ON co.order_id = o.order_id
```

```
GROUP BY s.seller_id, s.seller_name
ORDER BY total_products_sold DESC;
```

seller_id	seller_name	total_products_sold
114	Walton	7
111	Singer	4
100	Daraz	2
104	Pickaboo	2
109	Pickaboo	2
112	Apex	2
117	Chaldal	1

11. Sellers with min price < 200 and max price > 600

```
SELECT s.seller_id, s.seller_name, MIN(p.price) AS
min_price, MAX(p.price) AS max_price
FROM Sellers s
JOIN Products p ON s.seller_id = p.seller_id
GROUP BY s.seller_id, s.seller_name
HAVING MIN(p.price) < 200 AND MAX(p.price) > 600
ORDER BY s.seller_id;
```

seller_id	seller_name	min_price	max_price
109	Pickaboo	179.13	838.05
112	Apex	79.02	961.70

Hard Queries

Correlated Subqueries

12. Most recent order date per customer

```
SELECT c.customer_id, c.name,
  (SELECT MAX(o.order_date) FROM Orders o WHERE
o.customer_id = c.customer_id) AS last_order
FROM Customers c
LIMIT 5;
```

customer_id	name	last_order
100	Sabbir Ahmed	NULL
101	Md Abdullah	2024-09-20
102	Shovon Debnath	NULL
103	Rahim Uddin	2020-06-27
104	Karim Ahmed	2020-10-02

Subqueries inside WHERE

13. Top 3 products sold only in Delivered/Processing orders

```
SELECT
    p.product_id,
    p.product_name,
    SUM(p.price) AS total_sales
FROM Products p
JOIN Contains co ON p.product_id = co.product_id
WHERE co.order_id IN (
    SELECT order_id
    FROM Orders
    WHERE order_status IN ('Delivered', 'Processing')
)
GROUP BY p.product_id, p.product_name
ORDER BY total_sales DESC
LIMIT 3;
```

```
+------------+--------------+-------------+
| product_id | product_name | total_sales |
+------------+--------------+-------------+
|        107 | Backpack     |     1457.56 |
|        113 | T-Shirt      |     1215.54 |
|        105 | Laptop       |      841.81 |
+------------+--------------+-------------+
```

Comparing Aggregates with Global Aggregates

14. Sellers whose average price > overall average price

```
SELECT s.seller_id, s.seller_name, AVG(p.price) AS
seller_avg_price
FROM Sellers s
JOIN Products p ON s.seller_id = p.seller_id
GROUP BY s.seller_id
HAVING AVG(p.price) > (SELECT AVG(price) FROM
Products);
```

```
+-----------+-------------+------------------+
| seller_id | seller_name | seller_avg_price |
+-----------+-------------+------------------+
|       114 | Walton      |       516.562500 |
|       112 | Apex        |       520.360000 |
|       111 | Singer      |       536.376667 |
|       104 | Pickaboo    |       698.825000 |
|       100 | Daraz       |       728.780000 |
|       107 | Chaldal     |       535.560000 |
|       109 | Pickaboo    |       508.590000 |
+-----------+-------------+------------------+
```

Distinct Count Subqueries

15. Customers who ordered from more than two sellers

```
SELECT c.customer_id, c.name
FROM Customers c
WHERE (SELECT COUNT(DISTINCT o.seller_id)
FROM Orders o WHERE o.customer_id = c.customer_id) >
2;
```

```
+-------------+-------------+
| customer_id | name        |
+-------------+-------------+
|         101 | Md Abdullah |
+-------------+-------------+
```

Group-Based Filtering Using Subqueries

16. Customers who placed more orders than the average customer

```
SELECT c.customer_id, c.name
FROM Customers c
WHERE (SELECT COUNT(*)
      FROM Orders o
      WHERE o.customer_id = c.customer_id)
      >
      (SELECT AVG(order_count)
      FROM (
            SELECT COUNT(*) AS order_count
            FROM Orders
            GROUP BY customer_id
           )
         AS temp);
```

customer_id	name
101	Md Abdullah
105	Rafiul Islam
106	Nusrat Jahan
109	Tanvir Hossain
118	Farzana Karim
119	Samiul Hossain

Conclusion

The Online Shopping Management System designed in this project successfully illustrates how a relational database can support the core operations of an e-commerce platform. By implementing a collection of structured MySQL tables with clear relationships, the system captures essential activities such as customer management, product organization, seller information handling, order processing, payment tracking, and shipping updates. The database demonstrates how well-designed schemas and constraints can bring order and consistency to large, frequently changing datasets typical of online retail systems.

Key Features

This database system supports several important features:

1. **Customer, Seller, and Product Management** – Maintains customer details, seller records, and a categorized product catalog with pricing.

2. **Order and Payment Processing** – Handles multi-item orders, tracks order status, and stores payment mode, status, and transaction details.

3. **Inventory and Warehouse Tracking** – Monitors stock levels, warehouse information, and product restocking.

4. **Shipping and Delivery Management** – Records shipping dates, delivery progress, and any return-related updates.

Future Enhancements

To improve the system and make it more suitable for real-world commercial use, several enhancements can be added:

1. **User Authentication & Frontend Integration** – Secure login for customers, sellers, and admins, fully connected to a web or mobile interface.

2. **Product Interaction Features** – Support for customer reviews, ratings, and personalized product recommendations.

3. **Sales & Promotion Tools** – Include coupon codes, discounts, and a seller analytics dashboard for performance insights.

4. **Operational Enhancements** – Real-time delivery tracking, automated stock alerts, and efficient seller-side inventory management.

CEP Mapping

Our project is a solution to a complex engineering problem because it can't be resolved without in depth engineering knowledge. There is no obvious solution to and requires someamountofabstract thinking depending on the Database model. It also involves a diverse group of stakeholders with widely varying needs.

How the Knowledge Profiles (Ks) Are Addressed Through Our Project

Ks	Attribute	How Ks Are Addressed Through the Project	COs	POs
K3	Engineering Fundamentals	The project required understanding of database fundamentals to formulate the model using MySQL server, including relational tables like Products and Orders for efficient data management.	CO1, CO2, CO3	PO1, PO2
K4	Specialist Knowledge	Specialized knowledge of e-commerce operations, such as inventory tracking and payment processing, was needed, drawing from trade and business principles similar to export-import contexts.	CO6, CO7	PO6, PO8
K5	Engineering Design	ER Diagram and Schema Diagram were used to create the model, defining relationships like foreign keys between Orders and Customers or Products and Category.	CO3, CO4	PO3, PO2
K6	Engineering Practice	Use of modern tools like online compilers like OneCompiler and MySQL for implementing tables, inserting data, and running queries (e.g., joins between Shipping and Orders).	CO1, CO2, CO5	PO1, PO5
K7	Comprehension	Allows people in society to directly buy goods or services over the internet from the comfort of their homes or workplaces, impacting e-commerce growth and accessibility while considering data security.	CO6, CO7	PO6, PO8

How the Complex Engineering Problems (Ps) Are Addressed Through Our Project and Mapping Among Ps, COs, POs

Ps	Attribute	How Ps Are Addressed Through the Project	COs	POs
P1	Depth of Knowledge Required	Cannot be resolved without in-depth knowledge of engineering at the level of one or more Database fundamentals **(K3)**, ER diagram & Schema diagram **(K5)**, use of SQL server to implement SQL **(K6)**, and lastly the impact on society **(K7)**. This includes designing interconnected tables like Contains (linking Orders and Products) and ensuring data integrity via primary/foreign keys.	CO1, CO2, CO3, CO4, CO5	PO1, PO3, PO5, PO6, PO7, PO8
P3	Depth of Analysis Required	Extensive knowledge was required as the project contains some approaches which might have been done in another way, such as the ER model might not always be exactly the way that was shown in the project. So in-depth analysis would help to find or collect information from the database more efficiently, e.g., optimizing queries for Inventory restocking or Payment status checks.	CO3, CO4, CO5	PO6, PO7, PO8
P6	Extent of Stakeholder Involvement and Conflicting Requirements	Diverse groups of stakeholders were involved in the project, such as some customers in need of kitchen appliances, others needing new clothing, or electronic devices. Sellers manage products, and administrators handle shipping/delivery conflicts, reflecting varying needs like fast delivery vs. cost efficiency.	CO3, CO5, CO6	PO2, PO7
P7	Interdependence	The project allowed partitioning the whole problem into smaller sub problems. For example, if someone wanted to know about the products available in the system, the Products table was there to serve. Likewise, if someone wanted to know which orders were delivered to the customers, the Orders table was there to serve, with links to Shipping and Payments for integrated tracking.	CO3, CO4, CO5, CO7, CO8	PO2, PO7, PO11, PO12

How the Complex Engineering Activities (As) Are Addressed Through Our Project and Mapping Among As, COs, POs

As	Attribute	How As Are Addressed Through the Project	COs	POs
A1	Range of Resources	Diverse resources were required throughout the project. These include: → Information: Data of various types used in the database like Customers, Products, Orders etc. → Technology: Use of MySQL Server through OneCompiler to manage the database → Equipment: Personal Computer → People: Users, developers. → Money (e.g., for hosting or tools).	CO1, CO8, CO6	PO5, PO11
A4	Consequences for Society and Environment	The project would allow customers to search for and order products based on their need, admins will be able to keep track of the orders status and provide help if an order is not delivered, and also keep information of the shipments. This promotes sustainable e-commerce by reducing physical retail needs, though it raises environmental concerns like packaging waste.	CO6, CO7	PO6, PO8, PO12
A5	Familiarity	The project demonstrates familiarity with e-commerce management systems for various user types, including customers, sellers, and administrators, adapting concepts from related domains like export import for global scalability.	CO3, CO7, CO8	PO11, PO12

All the Files related to this project can be accessed in the link below

Drive Link